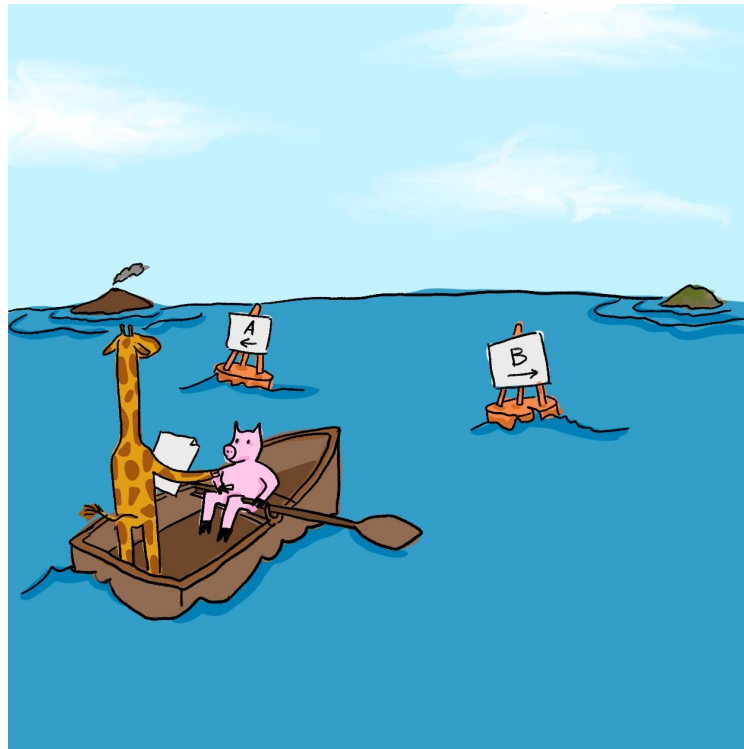


Introduction to Computability: Deterministic Finite Automata



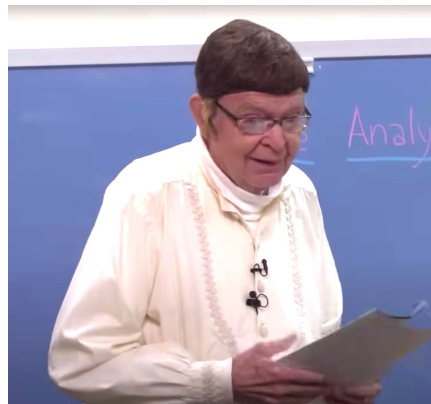
(art credit: CMU CS251)

“The question of whether a computer can think is no more interesting than the question of whether a submarine can swim.”
- Edsger Dijkstra

So far: Analysis of Algorithms

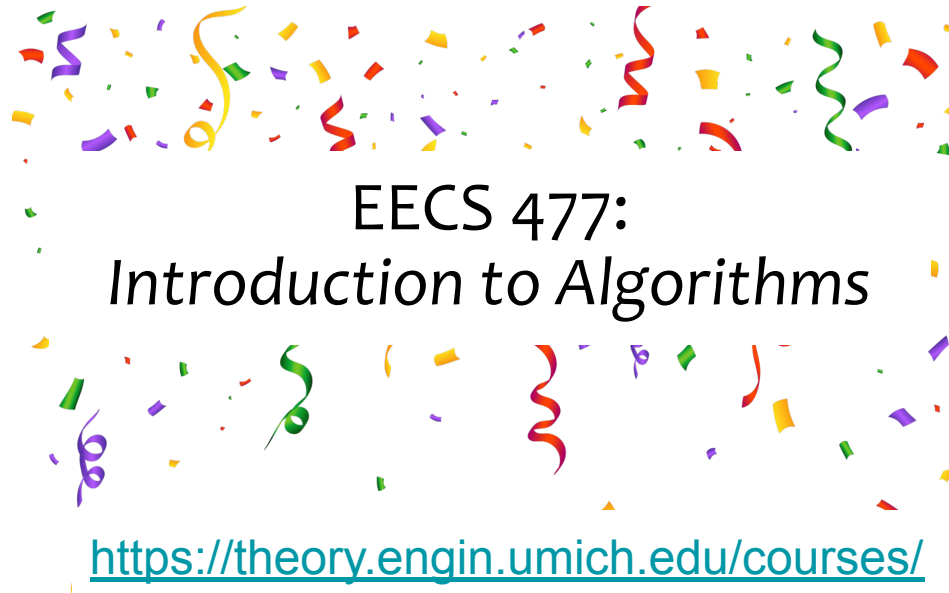


Don Knuth
“Father of Analysis of Algorithms”



You've just finished a crash course on algorithms!

If you found the material interesting, try:



Next up: **Computability**

- What is a computer?
- What problems can a computer solve?
- What problems can a computer **not** solve?

Techniques and Paradigms in this Course

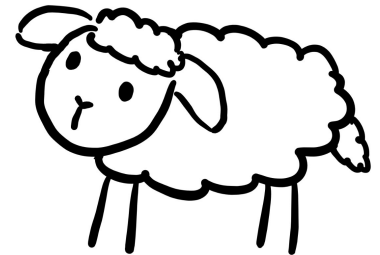
- Divide-and-conquer, greed, dynamic programming, the power of randomness
 Problems that are **easy** for a computer
- Computability  Problems that are **impossible** for a computer
- NP-completeness and approximation algorithms
- Cryptography  Problems that are "**probably hard**" for a computer
 Using "probably hard" problems for our benefit (hiding secrets)

Wait... what *is* a computer, anyway?

An algorithm solves a problem
if it gives the correct solution
on every instance.

Each of these
underlined words
has a precise
definition

We'll define last 3 terms now.
We'll save algorithm for later.



What is a computational **problem**?

We'll start with an example.

Example problem:

MULTIPLICATION

Instance

(input)

3, 7

610, 25

50, 610

15251, 252

12345679, 9

Solution

(output)

21

15250

30500

3843252

111111111

A **problem** is a collection of
instances and a **solution** to
each instance.

Example problem:

PALINDROME

Instance

(input)

a

10101

selfless

huh

376

emus sail i assume

Solution

(output)

Yes

Yes

No

Yes

No

Yes

PALINDROME is an example of a decision problem:

Problems where the solution is **Yes / No**.

(Also known as **True / False**,
1 / 0,
accept / reject.)

Representing problems

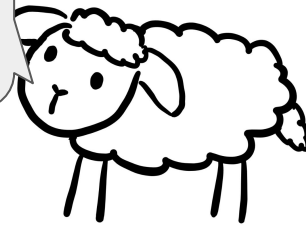
The instances of a problem can be (depending on the problem):

- numbers: 0, 1, 281, 376, ...
- strings: “hello”, “world”, “376”, ...
- pairs/tuples of numbers: (0,1), (3,7,6), ...
- lists of strings
- graphs
- images
- ...

These can all be conveniently encoded by **strings**.

Even just by **binary** (0/1) strings.

Binary is easy
when you only
have 2 “toes”
on each hoof!



String notation

Alphabet: A nonempty finite set Σ of symbols (characters).

$\Sigma = \{0,1\}$ is a popular choice.

String: A finite sequence of 0 or more symbols.

(or “word”)

The empty string (having zero symbols) is denoted ε .

For any $a \in \Sigma$:

a^k means k a 's

a^* means ≥ 0 a 's

a^+ means ≥ 1 a 's

Σ^k means all strings over Σ of length k .

Σ^* means **all** (finite) strings over Σ .

Σ^+ means all nonempty (finite) strings over Σ

For any x, y : $x|y$ means x OR y

Language: A collection (set) of strings.

I.e. any subset $L \subseteq \Sigma^*$.

The empty language is denoted $\emptyset$.

Examples of Languages

$$L = a^* = \{\epsilon, a, aa, aaa, \dots\}$$

Question: Does a^* have infinite-length strings? infinitely many strings?

Answer: No, and Yes: every individual string has *finite length*;
 a^* has *infinitely many* such strings.

$L = 01^* =$ set of (finite) strings of one 0 followed by zero or more 1's

$L = \{x^k y^k : k \geq 0\} =$ set of all strings consisting of some number of x's followed by the same number of y's

Question: $L = \{\epsilon\}$. Is $L = \emptyset$?

Answer: No. L has 1 element, $\emptyset$ has 0 elements

Question: How is $a^*|b^*$ different from $(a|b)^*$?

Representing problems

We can encode instances and solutions as strings.

Thus we can think of a **problem** as a **function**

$$f : \Sigma^* \rightarrow \Sigma^*$$

mapping instances to solutions.

So, a **decision problem** can be seen as a function

$$f : \Sigma^* \rightarrow \{\text{No}, \text{Yes}\}$$

Representing problems

A **decision problem** can be thought of as

$$f : \Sigma^* \rightarrow \{\text{No}, \text{Yes}\}$$

or equivalently as a **language**

$$L \subseteq \Sigma^*$$

where $L = \{x \in \Sigma^* : f(x) = \text{Yes}\}$, $f(x) = \begin{cases} \text{Yes} & \text{if } x \in L \\ \text{No} & \text{if } x \notin L \end{cases}$

E.g.: $L_{\text{PALINDROME}} = \{x \in \Sigma^* : x \text{ is a palindrome}\}$

What is **computation**?

What is an **algorithm**?

This lecture:

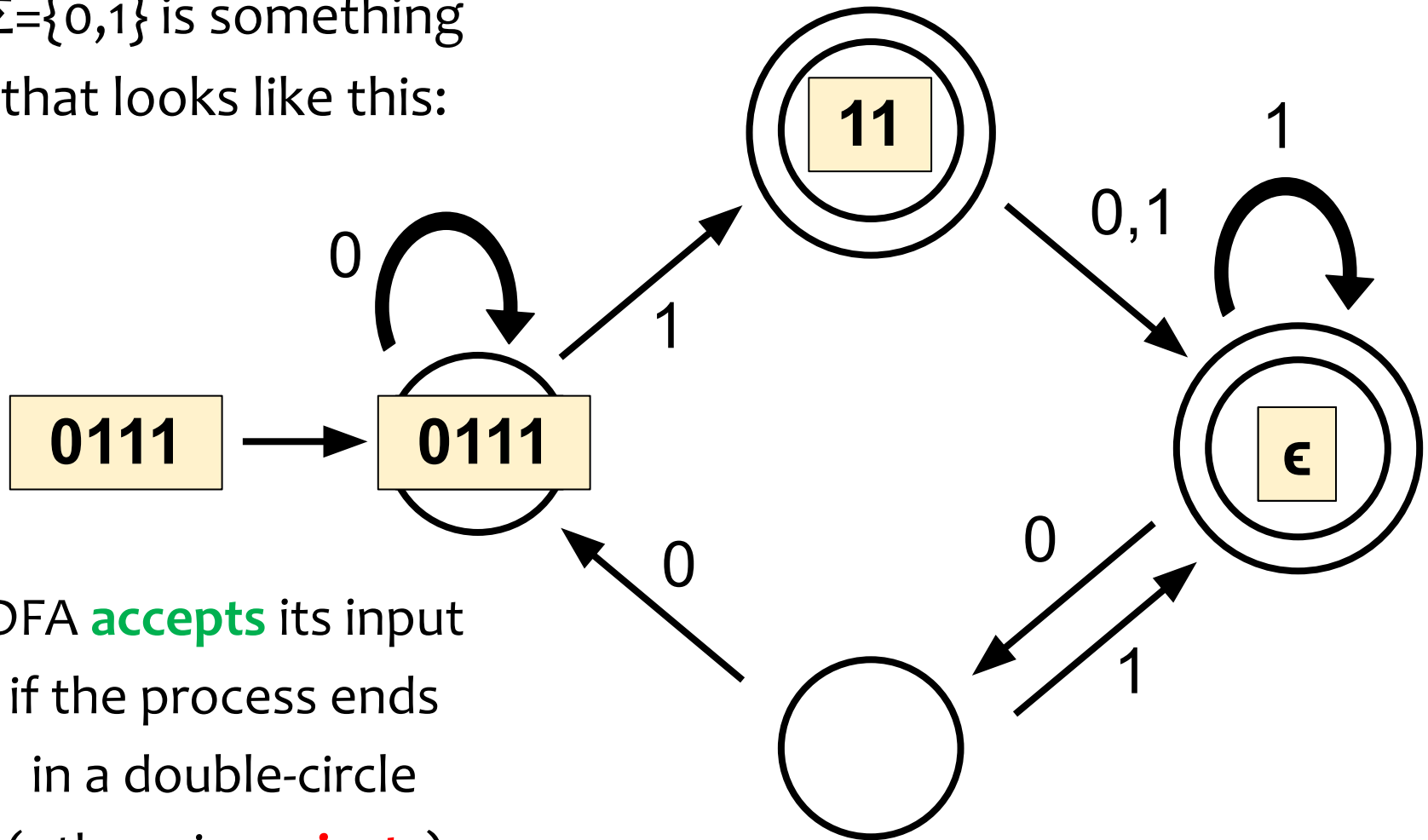
A restricted computational model:

Deterministic Finite Automata (DFA)

A good warmup before we study general models of computations next lecture.

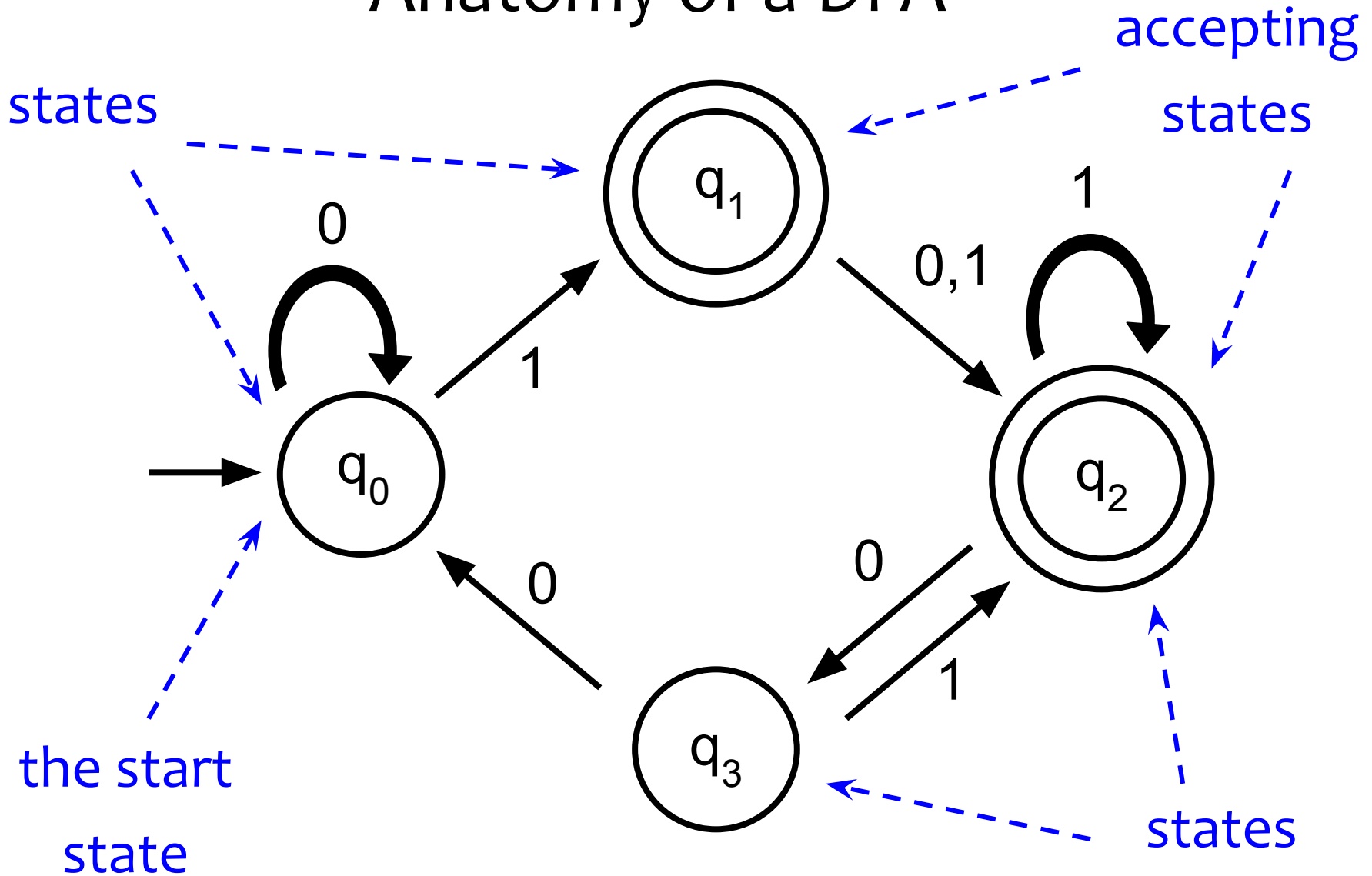
Deterministic Finite Automata (DFAs)

A DFA over alphabet $\Sigma=\{0,1\}$ is something that looks like this:



DFA **accepts** its input
if the process ends
in a double-circle
(otherwise **rejects**)

Anatomy of a DFA



transition rules: the labeled arrows
(one arrow per symbol for each state)

Computing with DFAs

Let M be a DFA, using alphabet Σ .

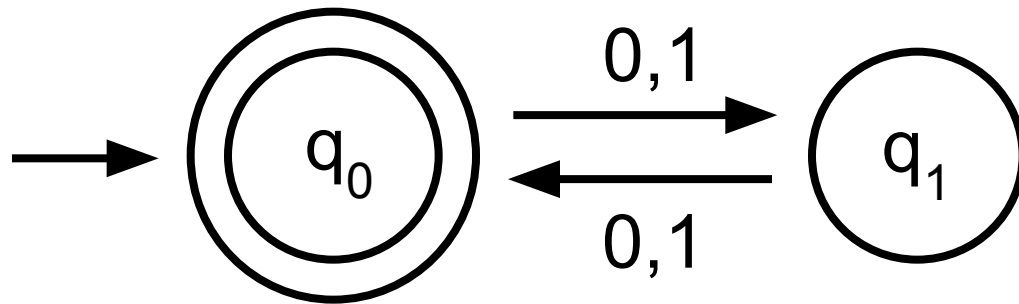
M **accepts** some strings in Σ^* , and **rejects** the rest.

Definition: $L(M) = \{x \in \Sigma^* : M \text{ accepts } x\}$

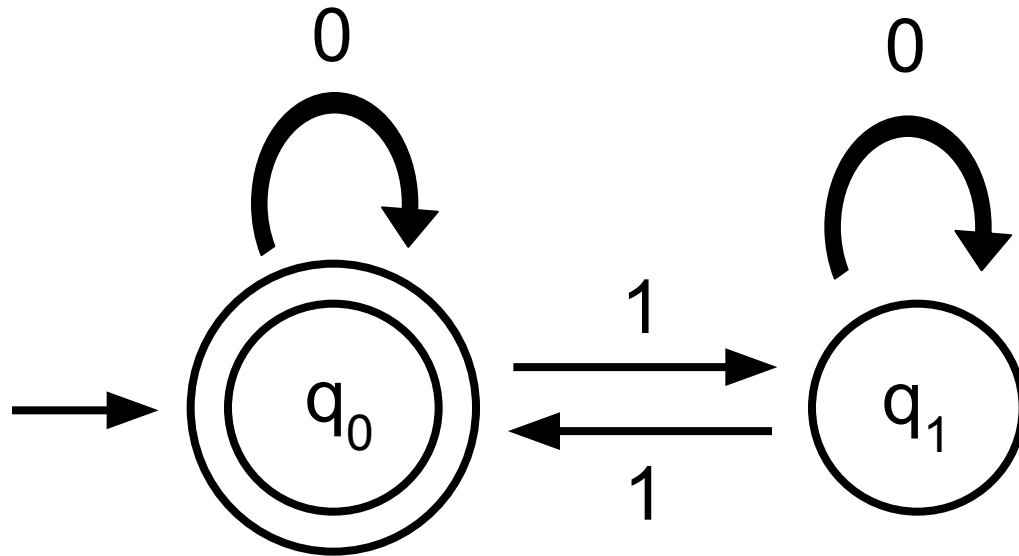
Called the “language decided by M ”.

If L is a language,

we say M **decides** L if $L(M) = L$.

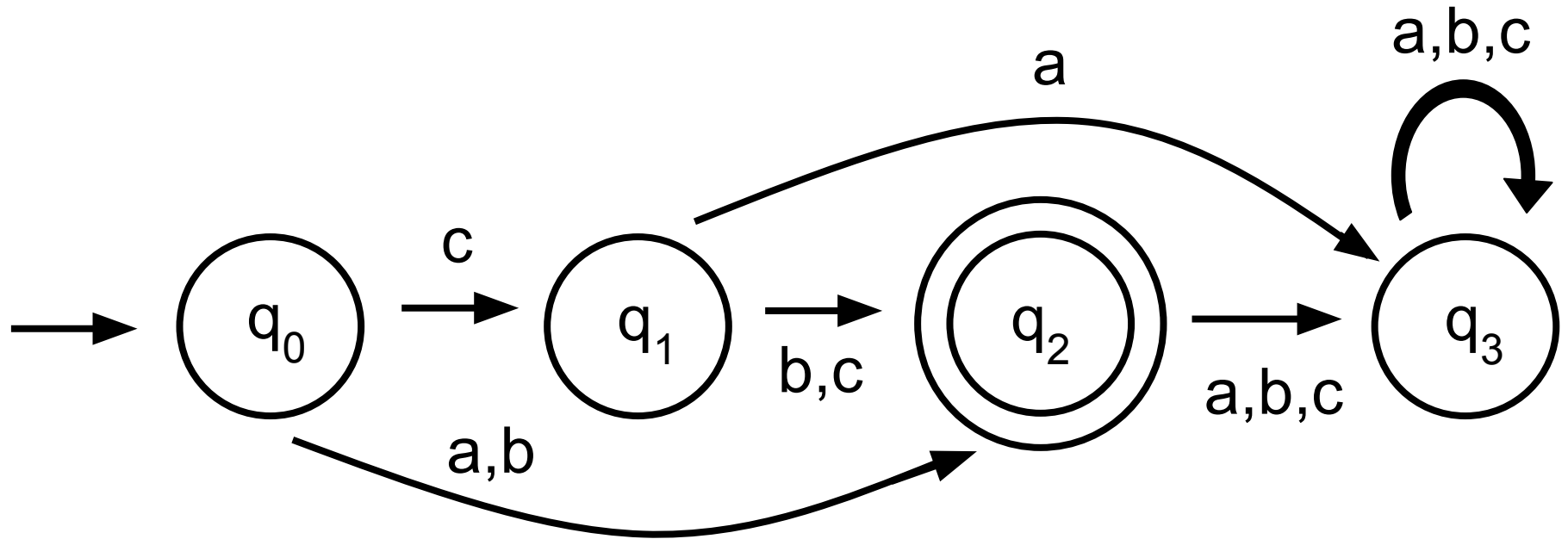


What language does this DFA decide?



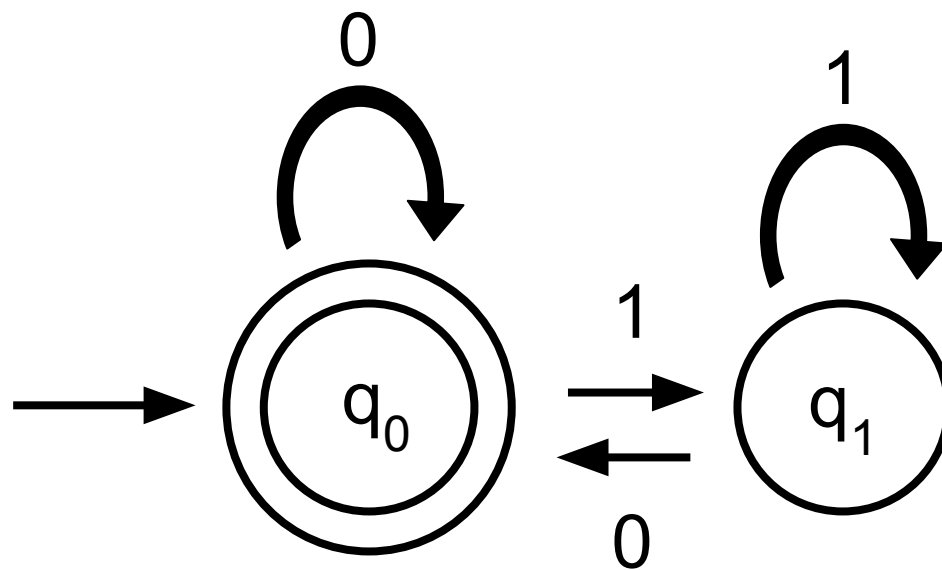
What language does this DFA decide?

M is the following DFA,
with alphabet $\Sigma=\{a,b,c\}$:



$L(M) =$

$M:$



$L(M) =$

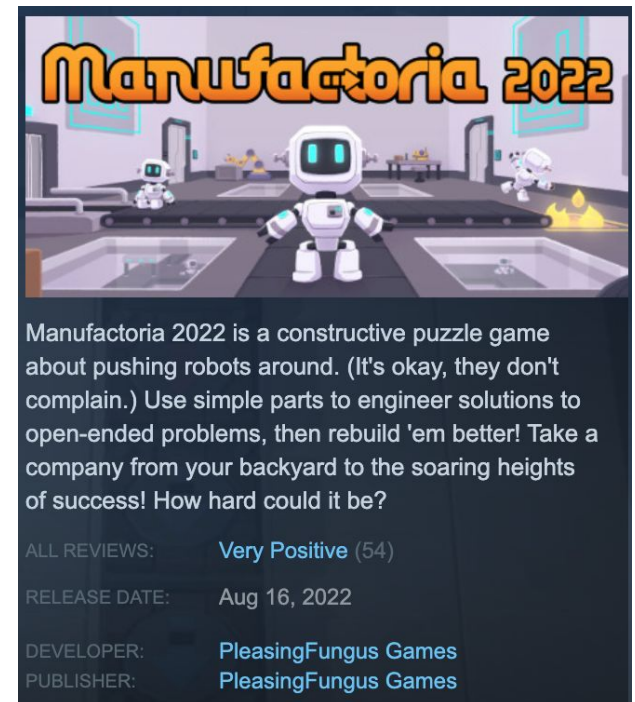
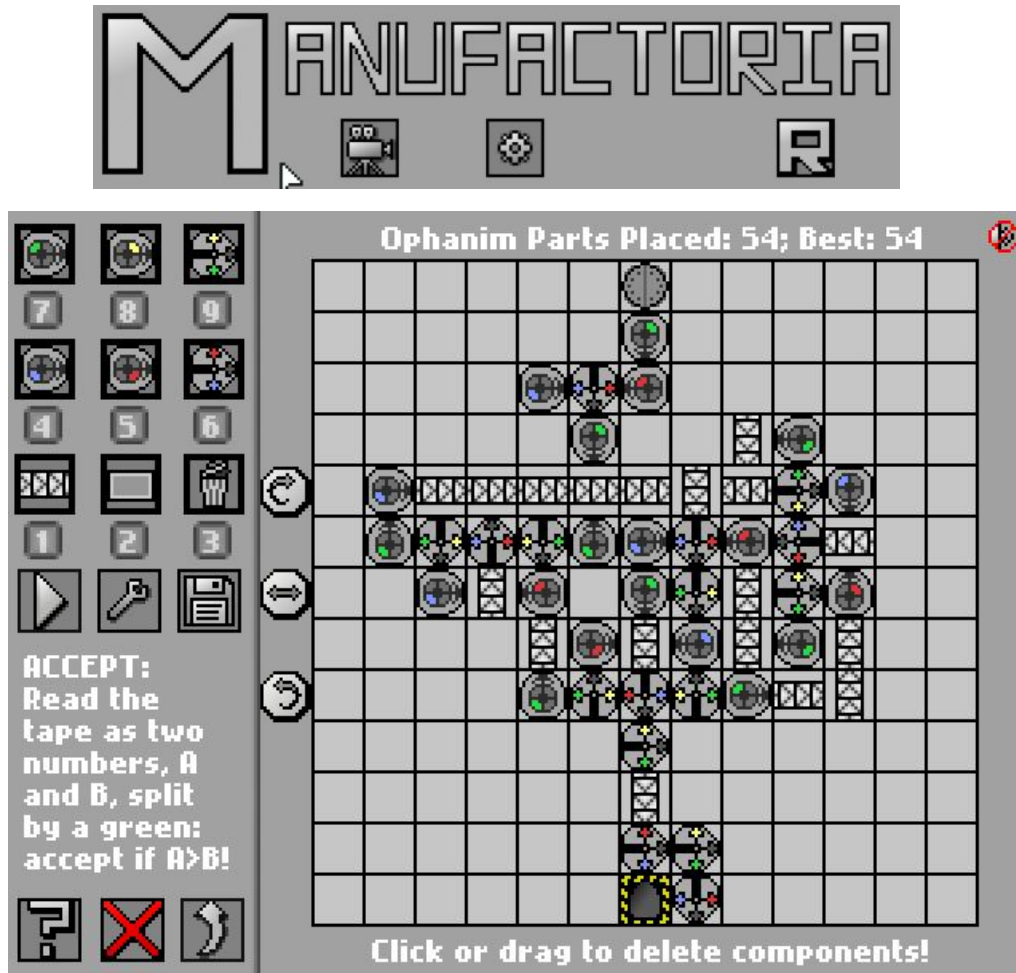
Fact: If there is a DFA that decides a language L , then there is also a DFA that decides L^c , the **complement of L** . Why?

E.g.

$L = \{\text{strings containing "01"}\}$

$L^c = \{\text{strings NOT containing "01"}\}$

What got me interested in CS in the first place...



Formal definition of DFAs

A **deterministic finite automaton** is a 5-tuple:

$$M = (\Sigma, Q, q_0, F, \delta)$$

Σ is a (finite) alphabet,

Q is a nonempty *finite* set of states,

$q_0 \in Q$ is the start state,

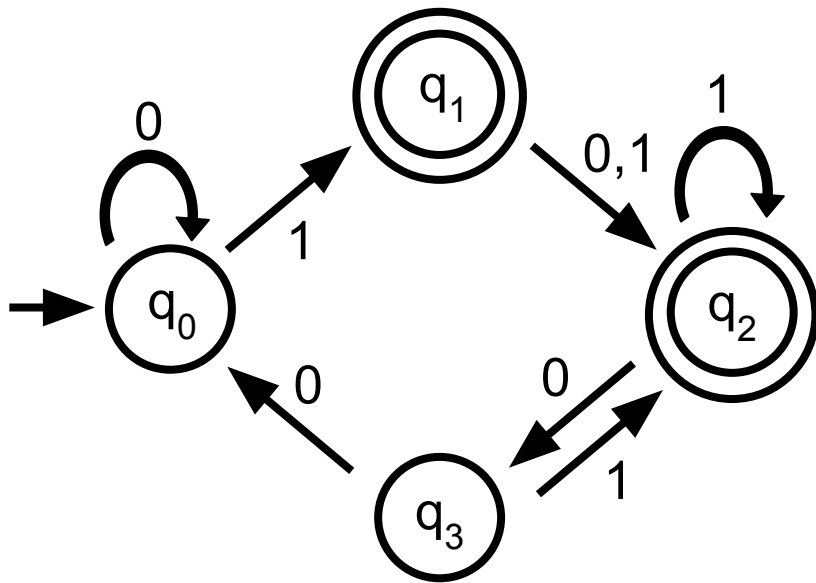
$F \subseteq Q$ is the set of accepting states,

$\delta : Q \times \Sigma \rightarrow Q$ is the state-transition function.

Formal definition of DFAs

A **deterministic finite automaton** is a 5-tuple:

$$M = (\Sigma, Q, q_0, F, \delta)$$



$$\Sigma = \{0,1\}$$

$$Q = \{q_0, q_1, q_2, q_3\}$$

q_0 is the start state

$$F = \{q_1, q_2\}$$

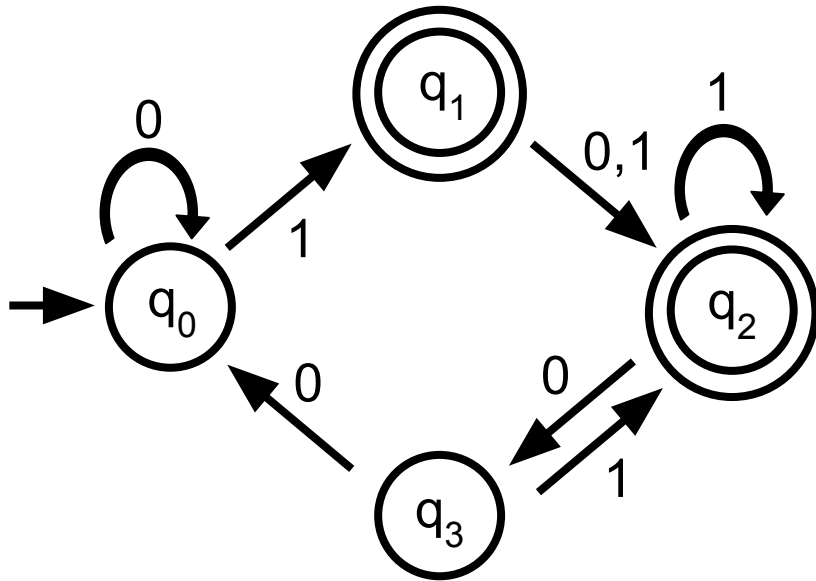
(δ on next slide)

Formal definition of DFAs

A **deterministic finite automaton** is a 5-tuple:

$$M = (\Sigma, Q, q_0, F, \delta)$$

$\delta : Q \times \{0,1\} \rightarrow Q$ is...



δ	0	1
q_0	q_0	q_1
q_1	q_2	q_2
q_2	q_3	q_2
q_3	q_0	q_2

1. For $\Sigma = \{a,b\}$, consider the language of strings with exactly one b
 - a. Write using string notation
 - b. Draw a DFA
2. Draw a DFA for $ab^*|ba^*$
3. Draw a DFA for $L = \{x^k y^k : k \geq 0\}$

This slide has $O(1)$ space for your work

Impossibility Proof Setup

Consider any DFA M . We'll show that M does not decide $L = \underline{\hspace{2cm}}$.

We'll exhibit a string on which M gives the “wrong” answer.

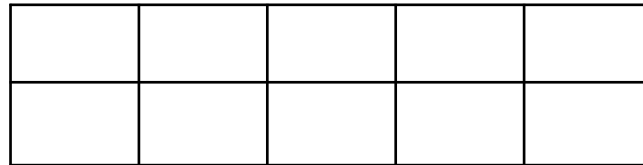
More specifically: We'll find 2 strings, $s_1 \in L$, $s_2 \notin L$, but prove that M does not have enough states to differentiate s_1 , s_2 and returns the same answer for both!

Impossibility Proof

Let s = #states of M (a finite number)

Consider the input string x^{s+1} .

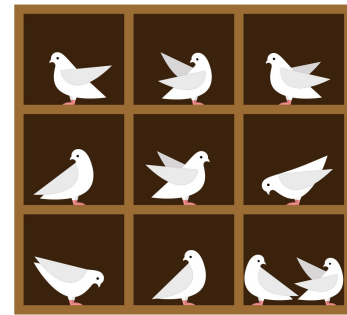
To process the string, the DFA visits some states:



There are $s+1$ transitions to process x^{s+1} , but M only has s states so there must be distinct $i, j \leq s+1$ such that $\text{state}(x^i) = \text{state}(x^j)$.

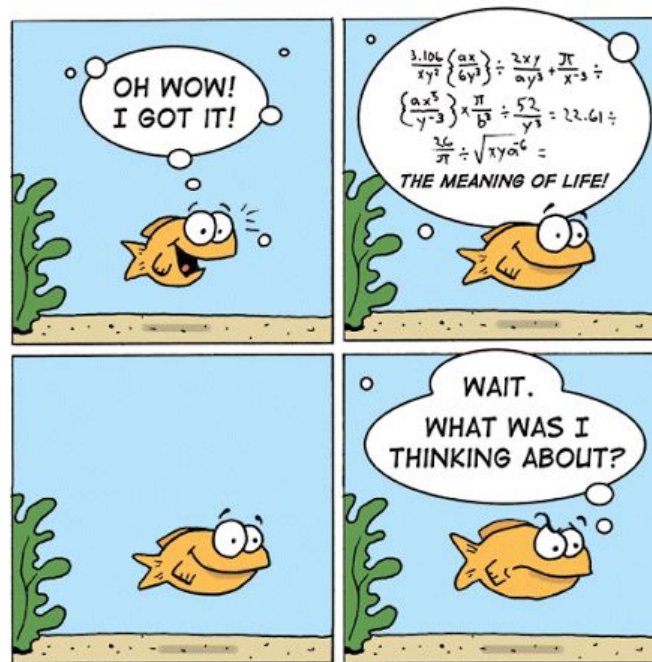
Claim. The two inputs $x^i y^i$ and $x^j y^i$ have the same final state. Why?

So M either accepts $x^i y^i$ and $x^j y^i$ or rejects both, so does not decide L .



Intuitive reason why DFAs cannot decide many languages:

Fixed amount of memory!



SPUDCOMICS.COM

© 2010 LONNIE EASTERLING

THE TRAGEDY OF A THREE SECOND MEMORY

A DFA models a computer with a fixed amount of memory
(think of the states as “memory”)

Real computers are only as powerful as DFAs.

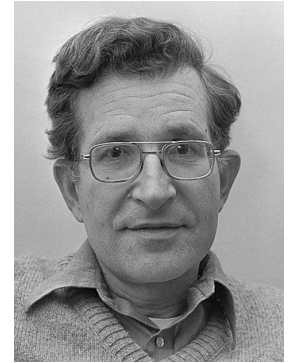
Your laptop also cannot decide the language $L = \{x^k y^k : k \geq 0\}$.

But, in our definition of a “computer”, we don’t want to artificially constrain ourselves to a fixed amount of memory.

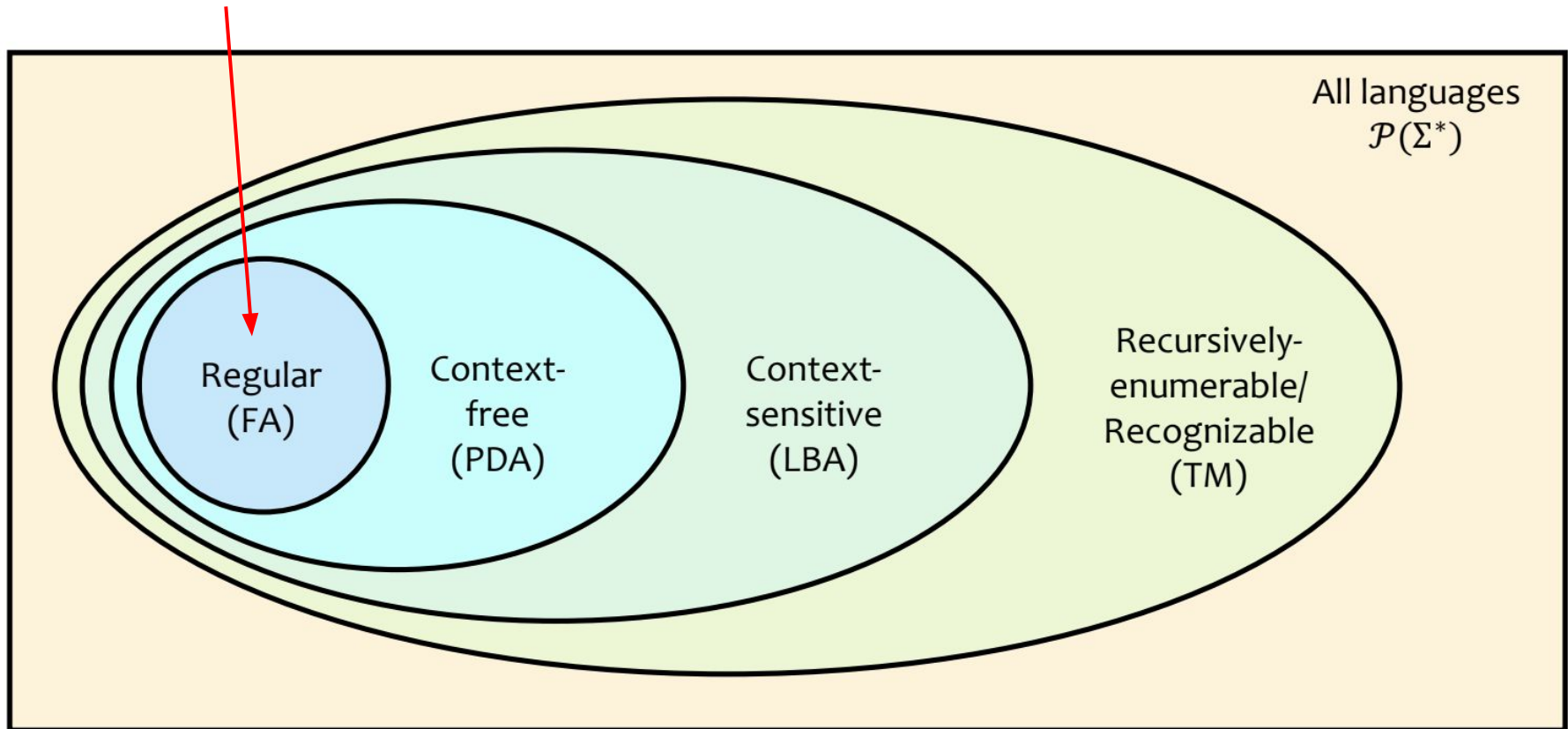
Instead, we want our definition of a “computer” to model the situation where you can always buy more memory if you need it.

That’s where **Turing Machines** come in.

The Chomsky Hierarchy (1956)



“Regular Language”: Language decidable by some DFA



More powerful memory system $\longrightarrow$

But DFAs are still used in practice:

- Simple household devices (switches, garage doors, ...)
- Software for designing and checking the behavior of digital circuits
- “Lexers,” the first stage of compilers
- Flexible string search in text files (logs, latex, ...)
 - **Warning:** “Regular expressions” in practice often *aren’t* regular!
- and much more...